



DISCLAIMER

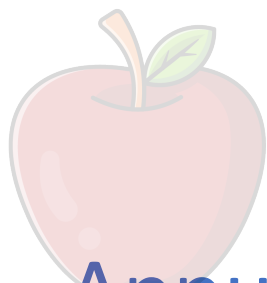
Il materiale contenuto nel drive è stato raccolto e richiesto tramite autorizzazione ai ragazzi frequentanti il corso di studi di Informatica dell'Università degli Studi di Salerno. Gli appunti e gli esercizi nascono da un uso e consumo degli autori che li hanno creati e risistemati per tanto non ci assumiamo la responsabilità di eventuali mancanze o difetti all'interno del materiale pubblicato.

Il materiale sarà modificato aggiungendo il logo dell'associazione, in tal caso questo possa recare problemi ad alcuni autori di materiale pubblicato, tale persona può contattarci in privato ed elimineremo o modificheremo il materiale in base alle sue preferenze.

Ringraziamo eventuali segnalazioni di errori così da poter modificare e fornire il miglior materiale possibile a supporto degli studenti.



CoScienze
Associazione



CoScienze

Associazione

Appunti ETC sul corso della Prof.ssa Luisa Gargano

ESERCIZI SVOLTI, DIMOSTRAZIONI E SPIEGAZIONI

SCRITTO DA MATTIA LIMONE

Indice

Dimostrazione unione di linguaggi regolari	2
Dimostrazione concatenazione di linguaggi regolari	3
Dimostrazione complementazione di linguaggio regolare	4
Dimostrazione intersezione di linguaggi regolari	5
Dimostrazione chiusura di Kleene per i linguaggi regolari	7
Definizione Ricorsiva per le Espressioni Regolari	8
Teorema di Kleene	8
Pumping Lemma	9
Definizione di configurazione MdT	9
Definizione di linguaggio riconosciuto da MdT	10
Dimostrazione per ogni DFA esiste una MdT equivalente	11
Dimostrazione debole della regolarità $L_1 \cup L_2$, $L_1 \circ L_2$, L_1^*	12
Dimostrazione complementazione di linguaggio ricorsivo	13
Dimostrazione complementazione di linguaggio ricorsivamente enumerabile	15
Definire una quintupla (NFA/DFA)	16
Determinare DFA a partire da un NFA	17
Determinare diagramma DFA con chiusura per l'intersezione	19
Determinare NFA da Espressione Regolare	21
Determinare Espressione Regolare da Automa	24
Funzione di transizione estesa F^*	26
Determinare configurazione MdT	27
Funzioni con MdT	29
Funzione di comparazione	29
Funzione Somma	32
Funzione Multiplo di 2	33
Risposte brevi a domande comuni	34

Dimostrazione unione di linguaggi regolari

L'unione di due linguaggi L_1 e L_2 è il linguaggio $L_1 \cup L_2$, costituito dalle parole appartenenti ad almeno uno fra L_1 e L_2 , cioè:

$$L_1 \cup L_2 = \{x \in \Sigma^* \mid x \in L_1 \vee x \in L_2\}$$

Consideriamo i due automi A_1 e A_2

$$A_1 = (Q_1, \Sigma_1, \delta_{N1}, q_{01}, F_1)$$

$$A_2 = (Q_2, \Sigma_2, \delta_{N2}, q_{02}, F_2)$$

che accettano i due linguaggi $L(A_1)$ e $L(A_2)$

allora si può costruire l'automa $A = (\Sigma, Q, \delta_N, q_0, F)$ che riconosce il linguaggio $L(A) = L(A_1) \cup L(A_2)$, che quindi è regolare:

- Si pone: $\Sigma = \Sigma_1 \cup \Sigma_2$.
Ovvero l'alfabeto A è l'unione degli alfabeti dei due automi operando.
- Si pone: $Q = Q_1 \cup Q_2 \cup \{q'_0\}$, dove q'_0 è un nuovo stato iniziale.
Gli stati di A sono gli stati dei due automi operando più un nuovo stato iniziale.
- Si pone: $F = F_1 \cup F_2$, oppure $F = F_1 \cup F_2 \cup \{q'_0\}$, se uno dei due automi riconosce anche la stringa vuota.
- La funzione di transizione δ_N viene definita come

$$\delta_N(q, a) = \delta_{N1}(q, a), \forall q \in Q_1, a \in \Sigma_1$$

$$\delta_N(q, a) = \delta_{N2}(q, a), \forall q \in Q_2, a \in \Sigma_2$$

$$\delta_N(q'_0, a) = \delta_{N1}(q_{01}, a) \cup \delta_{N2}(q_{02}, a), a \in \Sigma$$

L'automa A contiene, quindi, le transizioni definite per l'automa A_1 , quelle definite per A_2 ed inoltre le nuove transizioni che dallo stato iniziale q'_0 raggiungono gli stati raggiunti dalle transizioni definite per gli stati iniziali di A_1 e A_2 .

Dimostrazione concatenazione di linguaggi regolari

La concatenazione (o prodotto) di due linguaggi L_1 e L_2 è il linguaggio $L_1 \circ L_2$ delle parole scomponibili in una stringa di L_1 ed in una stringa di L_2 , cioè:

$$L_1 \circ L_2 = \{x \in \Sigma^* \mid \exists y_1 \in L_1, \exists y_2 \in L_2 (x = y_1 \circ y_2)\}$$

Consideriamo i due automi A_1 e A_2

$$A_1 = (Q_1, \Sigma_1, \delta_{N_1}, q_{0_1}, F_1)$$

$$A_2 = (Q_2, \Sigma_2, \delta_{N_2}, q_{0_2}, F_2)$$

- Si pone: $\Sigma = \Sigma_1 \cup \Sigma_2$.

Ovvero l'alfabeto A è l'unione degli alfabeti dei due automi operando.

- Si pone: $Q = Q_1 \cup Q_2$

- Si pone: $F = F_2$, oppure $F = F_1 \cup F_2$, se uno dei due automi riconosce anche la stringa vuota.

- Si pone $q_0 = q_{0_1}$

- La funzione di transizione δ_N viene definita come

$$\delta_N(q, a) = \delta_{N_1}(q, a), \forall q \in Q_1 - F_1, a \in \Sigma_1$$

$$\delta_N(q, a) = \delta_{N_2}(q, a), \forall q \in Q_2, a \in \Sigma_2$$

$$\delta_N(q, a) = \delta_{N_1}(q, a) \cup \delta_{N_2}(q_{0_2}, a), \forall q \in F_1, a \in \Sigma$$

Per costruzione, l'automa A accetta tutte e sole le stringhe di $L(A_1) \circ L(A_2)$.

Dimostrazione complementazione di linguaggio regolare

La complementazione di un linguaggio L , è il linguaggio $L_1 = \Sigma^* \setminus L$ costituito dalle parole appartenenti a Σ^* ma non ad L , cioè:

$$L_1 = \{x \in \Sigma^* \mid x \notin L\}$$

Considerato l'automa

$$A = (Q, \Sigma, \delta, q_0, F)$$

che riconosce $L = L(A)$ è possibile costruire l'automa

$$A = (Q \cup \{d \notin Q\}, \Sigma, \delta, q_0, \{Q - F \cup \{d\}\})$$

Come possiamo vedere dalla sua definizione, l'automa A ha le seguenti caratteristiche:

- L'alfabeto di input è uguale all'alfabeto di A .
- L'insieme degli stati è l'unione dell'insieme degli stati di A più un nuovo stato d .
- Lo stato iniziale è lo stato iniziale di A .
- Gli stati finali sono gli stati dell'automa A che non sono finali più lo stato d .
- La funzione δ è così definita:

$$\begin{aligned}\delta(q, a) &= \delta(q, a), \forall q \in Q, a \in \Sigma \\ \delta(q, a) &= d \text{ se } \delta(q, a) \text{ non è definito} \\ \delta(d, a) &= d, \forall a \in \Sigma\end{aligned}$$

Questa funzione definisce le transizioni che caratterizzano l'automa A , inoltre per ogni transizione di A non definita, la δ definisce una nuova transizione verso lo stato d .

Chiaramente ogni stringa che porta l'automa A in uno stato finale, porta l'automa A in uno stato non finale, e viceversa, ogni stringa che porta l'automa A in uno stato finale, porta l'automa A in uno stato non finale, per cui:

$$L(\bar{A}) = \Sigma^* - L(A) = \bar{L}(A)$$

Dimostrazione intersezione di linguaggi regolari

L'intersezione di due linguaggi L_1 e L_2 è il linguaggio $L_1 \cap L_2$ costituito dalle parole appartenenti ad L_1 e L_2 , cioè:

$$L_1 \cap L_2 = \{x \in \Sigma^* \mid x \in L_1 \wedge x \in L_2\}$$

Per dimostrare ciò basta considerare gli automi A_1 e A_2 , scelti in modo che riconoscano rispettivamente, i linguaggi L_1 e L_2 , ed osservare che, per le leggi di De Morgan, si ha che:

$$L(A_1) \cap L(A_2) \equiv \overline{\overline{L(A_1)} \cup \overline{L(A_2)}}$$

Siccome la classe dei linguaggi regolari è chiusa rispetto alle operazioni di unione e complementazione, $L_1 \cap L_2$ è regolare.

Procedendo invece per costruzione.

Consideriamo i due automi A_1 e A_2

$$\begin{aligned} A_1 &= (Q_1, \Sigma_1, \delta_{N1}, q_{01}, F_1) \\ A_2 &= (Q_2, \Sigma_2, \delta_{N2}, q_{02}, F_2) \end{aligned}$$

che accettano i due linguaggi $L(A_1)$ e $L(A_2)$

allora si può costruire l'automa $A = (\Sigma, Q, \delta_N, q_0, F)$ che riconosce il linguaggio $L(A) = L(A_1) \cap L(A_2)$, che quindi è regolare:

- Si pone: $\Sigma = \Sigma_1 \cup \Sigma_2$.
Ovvero l'alfabeto di A è l'unione degli alfabeti dei due automi operando.
- Si pone: $Q = Q_1 \times Q_2$
- Si pone: $F = F_1 \times F_2$
- Si pone: $q_0 = (q_{0_1}, q_{0_2})$
- La funzione di transizione δ_N viene definita come

$$\delta_N((q_1, q_2), a) = (\delta_{N_1}(q_1, a), \delta_{N_2}(q_2, a)), \forall q_1 \in Q_1, \forall q_2 \in Q_2, a \in \Sigma$$

$$\delta_N((q_{f1}, q_{f2}), a) = (\delta_{N_1}(q_{f1}, a), \delta_{N_2}(q_{f2}, a)), \forall q_{f1} \in F_1, \forall q_{f2} \in F_2, a \in \Sigma$$

Ogni stato dell'automata A è dunque composto da una coppia ordinata $(q_1 \in Q_1, q_2 \in Q_2)$ di stati che sono gli stati in cui si trovano rispettivamente gli automi A_1 e A_2 .

Affinché l'automata A accetti una stringa esso dovrà trovarsi in uno stato (q_{f1}, q_{f2}) dove $q_{f1} \in F_1$ e contemporaneamente $q_{f2} \in F_2$, di conseguenza l'automata accetta una stringa solo se essa è accettata da entrambi gli automi.

Dato che in grado di riconoscere il linguaggio $L_1 \cap L_2$; dal momento che per tale linguaggio esiste un accettore a stati finiti, possiamo concludere che $L_1 \cap L_2$ è regolare.

Dimostrazione chiusura di Kleene per i linguaggi regolari

Il linguaggio L^* definito da $L^* = \bigcup_{h=0}^{\infty} L^h$ dove $L^h = L \circ L^{h-1}$, $h \geq 1$ con la convenzione che $L^0 = \{\epsilon\}$, prende il nome di chiusura riflessiva e transitiva del linguaggio L rispetto all'operazione di concatenazione, mentre l'operatore $*$ prende il nome di iterazione o stella di Kleene.

Considerato l'automa

$$A = (Q, \Sigma, \delta, q_0, F)$$

che riconosce $L = L(A)$ è possibile costruire l'automa

$$A' = (Q \cup \{q'_0\}, \Sigma, \delta', q'_0, F \cup \{q'_0\})$$

che riconosce $L^* = (L(A))^*$ come segue:

- Alfabeto di input uguale all'alfabeto di A .
- Insieme degli stati uguale all'insieme Q più un nuovo stato iniziale q'_0 .
- Lo stato iniziale è il nuovo stato q'_0 .
- L'insieme degli stati finali è l'insieme F più lo stato q'_0 , in quanto il linguaggio L^* contiene la stringa vuota.
- La funzione di transizione δ' è così definita:

$$\delta'(q, a) = \delta(q, a), \forall q \in Q - F, a \in \Sigma$$

$$\delta'(q, a) = \delta(q, a) \cup \delta(q_0, a),$$

$$\delta'(q'_0, a) = \delta(q_0, a), a \in \Sigma$$

Per costruzione l'automa A' è in grado di accettare tutte e sole le stringhe di L^* , compresa la stringa vuota.

Definizione Ricorsiva per le Espressioni Regolari

Diciamo che R è un'espressione regolare se R è:

- α per qualche α nell'alfabeto X ,
- ε
- $\emptyset$
- $(R_1 \cup R_2)$, dove R_1 ed R_2 sono espressioni regolari,
- $(R_1 \circ R_2)$, dove R_1 ed R_2 sono espressioni regolari, o
- (R_1^*) , dove R_1 è un'espressione regolare.

Nei punti 1 e 2, le espressioni regolari α e ε rappresentano i linguaggi $\{\alpha\}$ e $\{\varepsilon\}$, rispettivamente. Nel punto 3, l'espressione regolare $\emptyset$ rappresenta il linguaggio vuoto. Nei punti 4, 5, e 6, le espressioni rappresentano i linguaggi ottenuti prendendo l'unione o la concatenazione dei linguaggi R_1 ed R_2 , o lo star del linguaggio R_1 , rispettivamente.

Teorema di Kleene

Un linguaggio L è regolare $\Leftrightarrow$ si può individuare con un'espressione regolare.

Dimostrazione

Sapendo che:

Linguaggio L è regolare $\Leftrightarrow L$ riconosciuto da NFA $\Leftrightarrow L$ riconosciuto da DFA

Si mostra che:

Per ogni DFA A possiamo costruire una e.r. R con $L(R) = L(A)$.

Per ogni e.r. R esiste un NFA A , tale che $L(A) = L(R)$. Cioè:

L riconosciuto da DFA $\Leftrightarrow L$ ammette un'e.r.

Quindi:

Linguaggio L è regolare $\Leftrightarrow L$ ammette un'e.r.

Pumping Lemma

Per ogni linguaggio regolare L , esiste una costante positiva p tale che per ogni stringa w in L di lunghezza $|w| \geq p$ esistono stringhe x, y, z tali che $w = xyz$ tali che:

- $|y| > 0$
- $|xy| \leq p$
- $xy^iz \in L \forall i \geq 0$

Definizione di configurazione MdT

Supponiamo di avere a, b e c in Γ , così come u e v in Γ^* e gli stati q_i e q_j . In tal caso $uaq_i b v$ e $uq_j a c v$ sono due configurazioni. Diciamo che

$uaq_i b v$ produce $uq_j a c v$

se nella funzione di transizione $\delta(q_i, b) = (q_j, c, L)$. Questo nel caso in cui la macchina di Turing effettua uno spostamento verso sinistra. Per lo spostamento a destra, diciamo che

$uaq_i b v$ produce $uacq_j v$

se $\delta(q_i, b) = (q_j, c, R)$.

Definizione di linguaggio riconosciuto da MdT

L'insieme di stringhe che una MdT, che chiamiamo M , accetta rappresenta il linguaggio di M , o il linguaggio riconosciuto da M , denotato con $L(M)$.

Un linguaggio può essere definito **Turing-riconoscibile** o anche **linguaggio ricorsivamente enumerabile** se esiste una Macchina di Turing che lo riconosce.

Quando una Macchina di Turing si ferma su ogni singolo input e non cicla mai, vengono chiamate **decisori**, quindi tale macchina “**decide**” un certo linguaggio se riconosce tale linguaggio.

Un linguaggio si dice **Turing-decidibile** o anche **linguaggio ricorsivo** o semplicemente decidibile se esiste una macchina di Turing che lo decide.



Dimostrazione per ogni DFA esiste una MdT equivalente

Per ogni Automa Finito Deterministico esiste una Macchina di Turing equivalente

Dimostrazione

Sapendo che:

Linguaggio L è regolare $\Leftrightarrow L$ riconosciuto da NFA $\Leftrightarrow L$ riconosciuto da DFA

Si mostra che:

Presa una stringa χ posso sempre decidere se appartiene o meno al linguaggio L , perché posso sempre farla “riconoscere” o “non riconoscere” dall’automa di L , allora il linguaggio L è decidibile.

Linguaggio L è decidibile $\Leftrightarrow L$ deciso da MdT

Il linguaggio L regolare è decidibile e quindi esiste una Macchina di Turing equivalente.

Se un linguaggio è decidibile allora esiste una macchina di Turing che lo riconosce. Quindi il linguaggio è Turing riconoscibile.

Considerato l’automa

$$A = (Q, \Sigma, \delta_1, q_{0_1}, F)$$

che riconosce $L = L(A)$ è possibile costruire la macchina di Turing

$$M = (Q, \Sigma, \Gamma, \delta_2, q_{0_2}, q_{accept}, q_{reject})$$

che riconosce lo stesso linguaggio come segue:

- Alfabeto di input uguale all’alfabeto di A
- $\Gamma = \Sigma \cup \{\underline{b}\}$ insieme dell’alfabeto di A più il simbolo di blank
- Insieme degli stati uguale all’insieme degli stati di A

- $q_{0_1} = q_{0_2}$ come stato iniziale
- q_{accept} come lo stato di accettazione legato tramite transizione definita da δ_2 dagli stati finali A
- q_{reject} come lo stato di rigetto legato tramite transizione definita da δ_2 dagli stati non finali A
- δ_2 definita come segue:

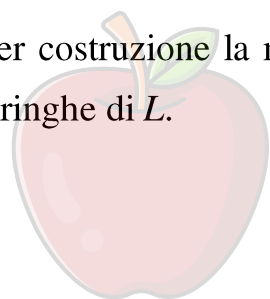
$$\delta_1(q, a) = q' \Rightarrow \delta_2(q, a) = (q', a, R), \forall q \in Q, \forall q' \in Q, a \in \Sigma$$

$$\delta_1(q, a) = q' \Rightarrow \delta_2(q, a) = \delta_2(q_{accept}, a, R), \forall q \in F, a \in \{\underline{b}\}$$

$$\delta_1(q, a) = q' \Rightarrow \delta_2(q, a) = \delta_2(q_{reject}, a, R), \forall q \notin F, a \in \{\underline{b}\}$$

Si nota che la seconda e la terza definizione definiscono la transizione verso gli stati q_{accept} e q_{reject} .

Per costruzione la macchina di Turing M è in grado di accettare tutte e sole le stringhe di L .



CoScienze
Associazione

Dimostrazione debole della regolarità $L_1 \cup L_2$,

$$L_1 \circ L_2, L_1^*$$

Siano L_1 e L_2 linguaggi regolari, allora anche $L_1 \cup L_2, L_1 \circ L_2, L_1^*$ sono linguaggi regolari.

Dimostrazione

Dato che L_1 e L_2 sono linguaggi regolari allora esistono espressioni regolari E ed F tali che $L_1 = L(E)$ e $L_2 = L(F)$.

Allora $L(E + F) = L_1 \cup L_2$, $L(E \circ F) = L_1 \circ L_2$, $L(E^*) = L_1^*$ sono regolari per definizione.

Dimostrazione complementazione di linguaggio ricorsivo

La complementazione di un linguaggio L_1 è il linguaggio $L_1 = \Sigma^* \mid L_1$ costituito dalle parole appartenenti a Σ^* ma non ad L_1 , cioè:

$$L_1 = \{x \in \Sigma^* \mid x \notin L_1\}$$

Considerata la macchina di Turing che decide L_1 e che rigetta $\bar{L}_1$ (come da def.)

$$A = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept_1}, q_{reject_1})$$

è possibile costruire la macchina di Turing che decide $\bar{L}_1$ e che rigetta L_1

$$A = Q, \Sigma, \Gamma, \delta, q_0, q_{accept_2}, q_{reject_2}$$

Come possiamo vedere dalla sua definizione, la macchina di Turing A ha le seguenti caratteristiche:

- L'alfabeto di input è uguale all'alfabeto di A
- L'insieme degli stati è uguale all'insieme degli stati di A
- Lo stato iniziale è lo stato iniziale di A
- Gli stati di q_{accept} diventano gli stati di q_{reject} e viceversa come definiti dalla funzione di transizione
- La funzione δ è così definita:

$$\delta(q, a) = q, a, s \Rightarrow \delta(q, a) = q, a, s, \forall q \in Q, a \in \Gamma, s \in \{L, R\}$$

$$\delta(q, a) = q_{accept_1}, a, s \Rightarrow \delta(q, a) = q_{reject_2}, a, s, \forall q \in Q, a \in \Gamma, s \in \{L, R\}$$

$$\delta(q, a) = q_{reject_1}, a, s \Rightarrow \delta(q, a) = q_{accept_2}, a, s, \forall q \in Q, a \in \Gamma, s \in \{L, R\}$$

Considerato che una macchina di Turing che decide un linguaggio si ferma su ogni input e non va mai in loop, si può sempre costruire una macchina ad essa complementare invertendo gli stati di accettazione e rigetto.

Chiaramente ogni stringa che porta l'automa A in uno stato di accettazione, porta l'automa A in uno stato di rigetto, e viceversa, ogni stringa che porta

l'automa A in uno stato di accettazione, porta l'automa A in uno stato di rigetto, per cui:

$$L(\bar{A}) = \Sigma^* - L(A) = \overline{L(A)}$$



Dimostrazione complementazione di linguaggio ricorsivamente enumerabile

La complementazione di un linguaggio L_1 è il linguaggio $\bar{L}_1 = \Sigma^* \setminus L_1$ costituito dalle parole appartenenti a Σ^* ma non ad L_1 , cioè:

$$\bar{L}_1 = \{x \in \Sigma^* \mid x \notin L_1\}$$

Considerata la macchina di Turing che riconosce L_1

$$A = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept_1}, q_{reject_1})$$

non è possibile costruire la macchina di Turing

$$A = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept_2}, q_{reject_2})$$

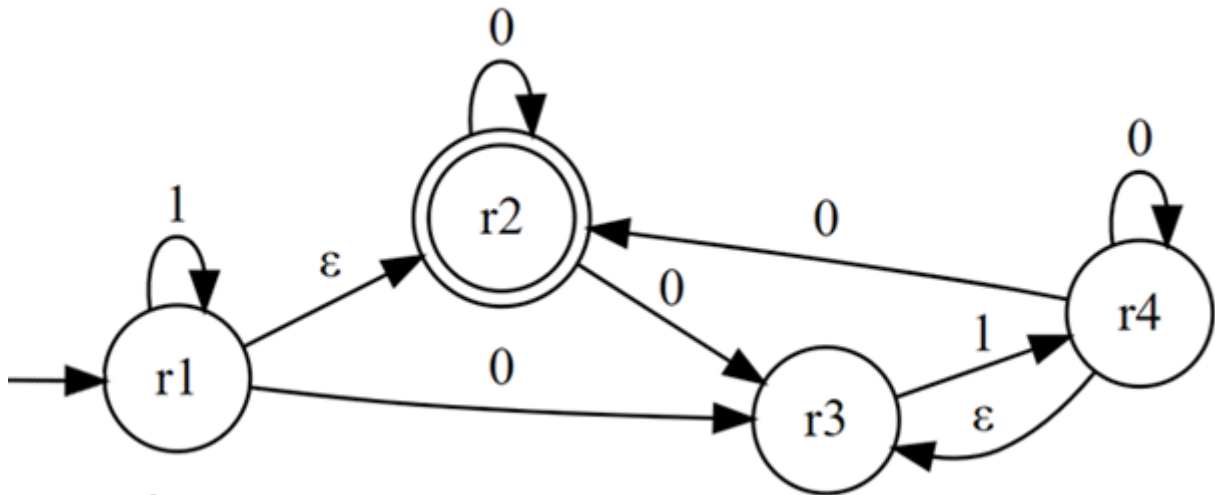
Al fine di costruire una macchina di Turing complementare, dobbiamo far in modo tale che A riconosca il linguaggio $\bar{L}_1$.

Denominando con R l'insieme dei linguaggi Turing-riconoscibili allora deve valere $L_1 \in R \wedge \bar{L}_1 \in R$ ma ciò ricade nella definizione di linguaggio ricorsivo e non di linguaggio ricorsivamente enumerabile, di conseguenza un linguaggio ricorsivamente enumerabile non è chiuso rispetto al complemento.

Per essere più precisi tale chiusura non è presente proprio per la definizione di linguaggio ricorsivamente enumerabile che permette di rispondere alla sola domanda “la stringa $x \in L_1$?”, non si può stabilire a priori se la macchina terminerà l'elaborazione di una stringa che non appartiene al linguaggio in un numero finito di passi, per cui se la macchina di Turing fosse in grado di riconoscere sia se la stringa appartenga al linguaggio sia se appartenga al suo complemento i linguaggi L_1 e $\bar{L}_1$ non sono solo ricorsivi enumerabili, ma anche ricorsivi.

Definire una quintupla (NFA/DFA)

Una quintupla è definita da $(Q, \Sigma, \delta, q_0, F)$



Di conseguenza

$Q = \{r1, r2, r3, r4\}$

$\Sigma = \{0, 1\}$

$q_0 = r1$

$F = \{r2\}$

δ sarà definita dalla tabella

	ϵ	0	1
r1	$\{r2\}$	$\{r3\}$	$\{r1\}$
r2	Φ	$\{r2, r3\}$	Φ
r3	Φ	Φ	$\{r4\}$
r4	$\{r3\}$	$\{r4\}$	$\{r2\}$

Q rappresenta l'insieme degli stati

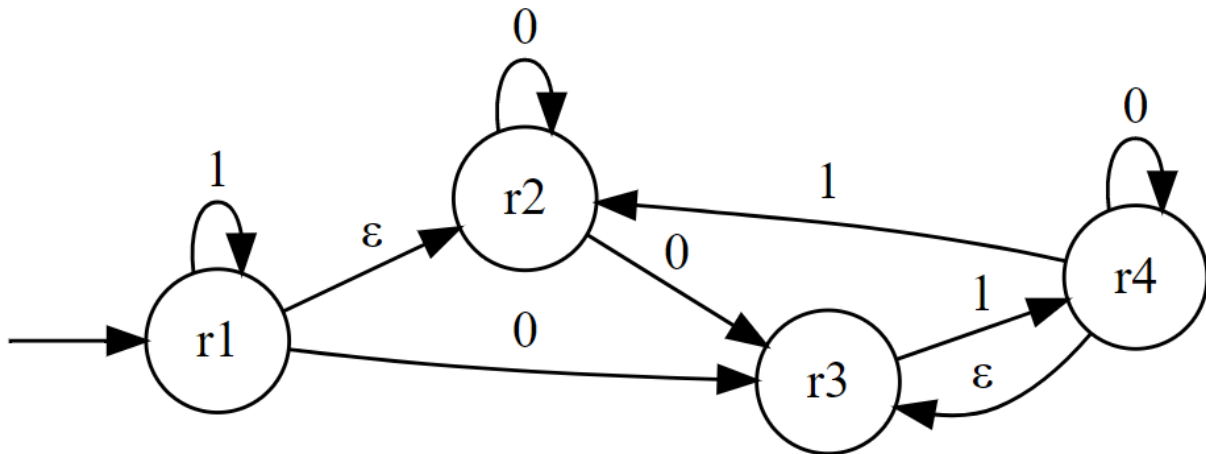
Σ l'alfabeto

q_0 lo stato iniziale

F l'insieme degli stati finali

δ la funzione di transizione

Determinare DFA a partire da un NFA



Prendiamo in esempio il seguente NFA

I passi da seguire sono i seguenti

- Determinare l'insieme degli stati iniziali
- Seguo il percorso degli insiemi di stati fino all'esaurimento dei percorsi
- Determino gli stati finali, scegliendoli tra quelli che presentano almeno uno stato finale dell'NFA

Di conseguenza lo stato iniziale sarà $\{r_1, r_2\}$ data la presenza dell' ϵ transition.

$\{r_1, r_2\}$ con 1 cicla su sé stesso

$\{r_1, r_2\}$ con 0 va in $\{r_2, r_3\}$

$\{r_2, r_3\}$ con 1 va in $\{r_3, r_4\}$, fare attenzione all' ϵ transition in r_4

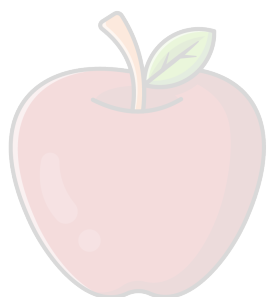
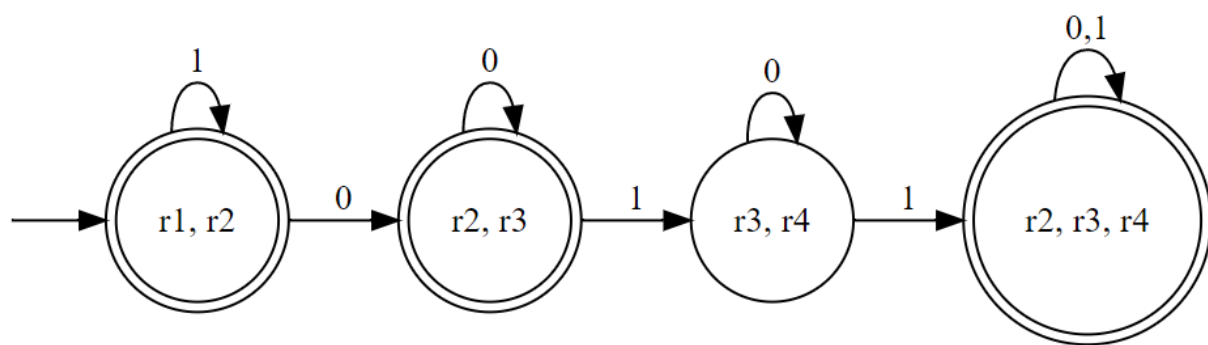
$\{r_2, r_3\}$ con 0 cicla su sé stesso

$\{r_3, r_4\}$ con 1 va in $\{r_2, r_3, r_4\}$ va in r_3 sempre per l' ϵ transition in r_4

$\{r_2, r_3, r_4\}$ con 1 cicla su sé stesso

$\{r_2, r_3, r_4\}$ con 0 cicla su sé stesso

Segno come stati finali quelli in cui è presente r_2 ed ottengo il seguente DFA



CoScienze
Associazione

Determinare diagramma DFA con chiusura per l'intersezione

Alfabeto: $\{a, b\}$

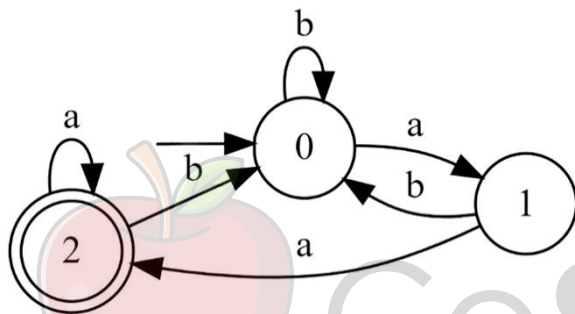
$L = \{w \mid w \text{ termina con } aa \text{ e non contiene sottostringa } bb\}$

1) Divido il problema in proprietà

Proprietà 1: $\{w \mid w \text{ termina con } aa\}$

Proprietà 2: $\{w \mid w \text{ non contiene sottostringa } bb\}$

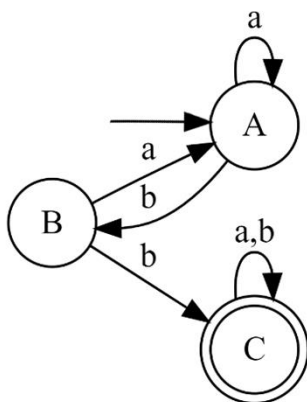
Automa per la proprietà 1



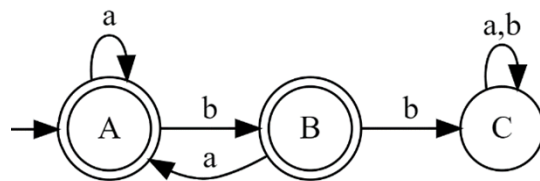
Automa per la proprietà 2

(Scrivo il complementare e poi inverte gli stati finali in non finali e non accettanti in finali)

Accetta bb

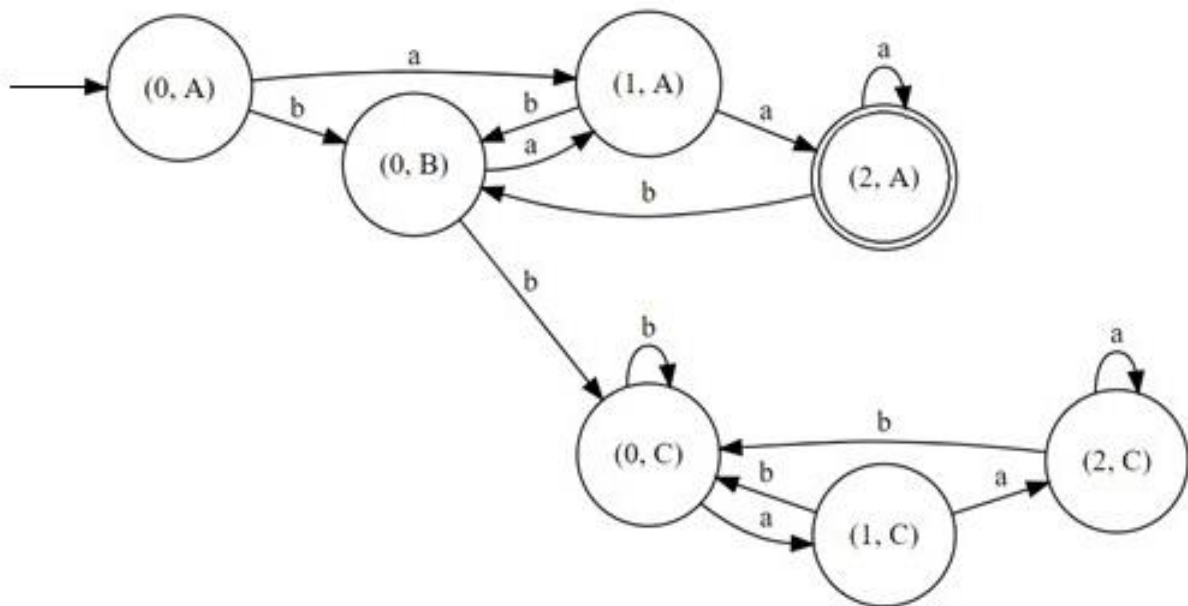


Non accetta bb



Effettuo a questo punto l'intersezione fra i due automi, si parte dalla coppia di stati iniziali e si analizza il comportamento al variare dell'alfabeto.

Automa intersezione



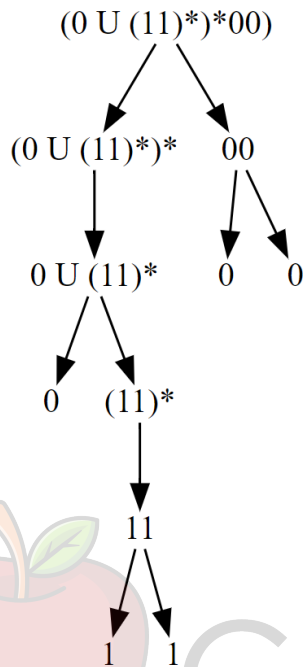
Ciò che ottengo è un automa di questo tipo, dove gli stati finali sono gli stati che contengono entrambi gli stati finali dei due automi di partenza (chiusura rispetto all'intersezione), quindi in questo caso avrebbero potuto essere solamente le coppie $(2, A)$ e $(2, B)$, ma il secondo non è presente.

Nel caso fosse richiesta la chiusura rispetto all'unione gli stati finali sarebbero allora stati quelli con almeno uno dei due stati finali dei precedenti automi $(2, x)$, (x, A) , (x, B) .

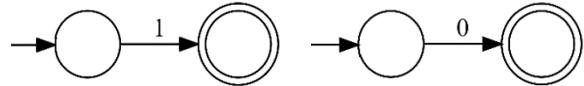
Determinare NFA da Espressione Regolare

Prendiamo in esempio la $RegEx = (0 \cup (11)^*)^*00$

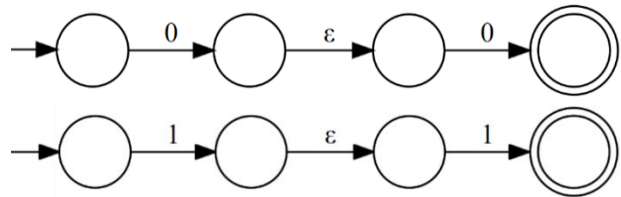
L'esercizio si svolge con la scomposizione dell'espressione regolare



Partiamo dagli automi 0 e 1 perché i più semplici



Risaliamo l'albero dell'espressione regolare e passiamo a 11 e 00

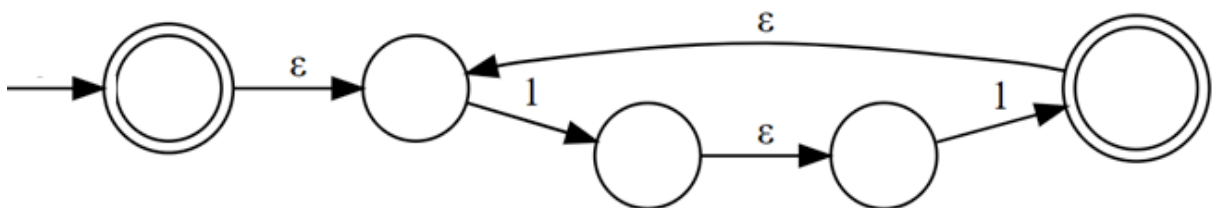


La procedura prevede di aggiungere una ϵ transition dallo stato finale del primo automa a quello iniziale dell'altro automa.

Adesso si passa a $(11)^*$.

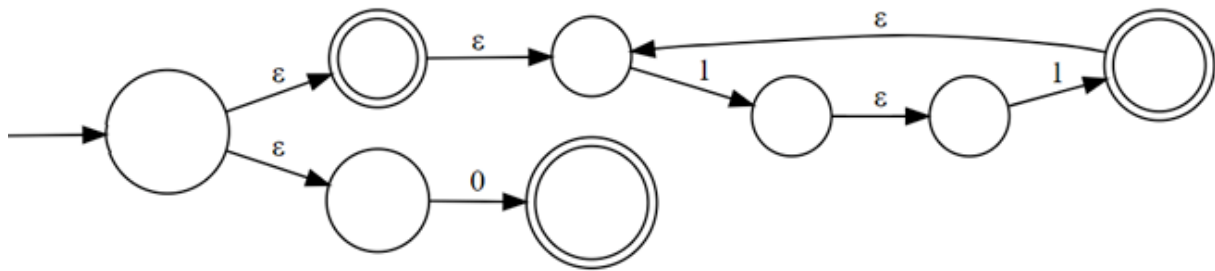
Per la chiusura di Kleene il passaggio è semplice:

- aggiungo un nuovo stato iniziale, che è anche finale per riconoscere la stringa vuota
- aggiungo una ϵ transition da questo nuovo stato iniziale al precedente stato iniziale
- aggiungo una ϵ transition dagli stati finali del precedente automa allo stato iniziale prima dell'aggiunta del nuovo stato iniziale



Adesso si passa a $0 \cup (11)^*$

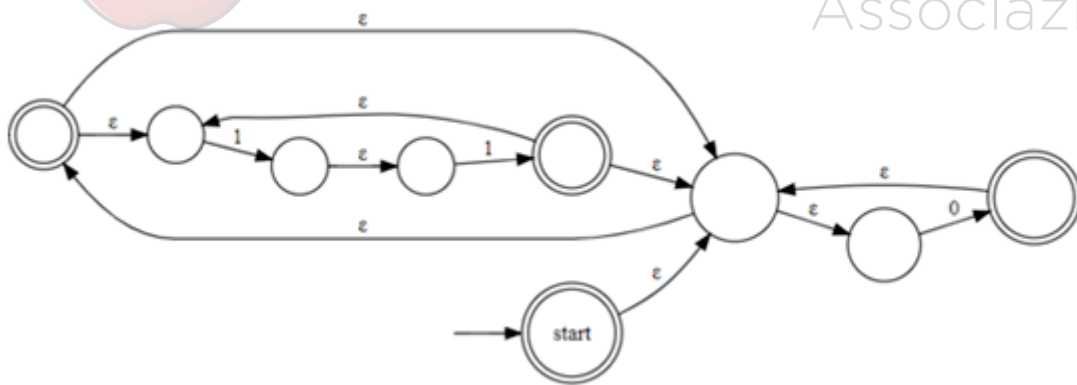
Per l'unione dato che abbiamo già generato i due automi da unire aggiungiamo un nuovo stato iniziale collegato con ϵ transition agli stati iniziali dei due automi da unire



Adesso si passa a $(0 \cup (11)^*)^*$

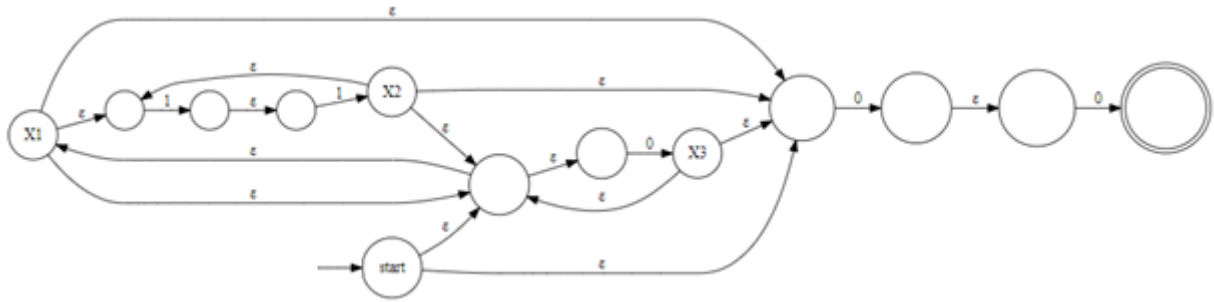
Il procedimento è lo stesso spiegato in precedenza, quindi:

- aggiungo un nuovo stato iniziale, che è anche finale per riconoscere la stringa vuota
- aggiungo una ϵ transition da questo nuovo stato iniziale al precedente stato iniziale
- aggiungo una ϵ transition dagli stati finali del precedente automa allo stato iniziale prima dell'aggiunta del nuovo stato iniziale



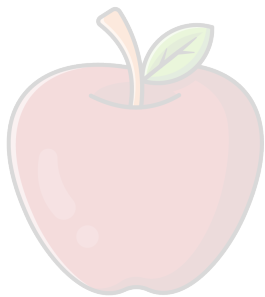
Adesso si passa a $(0 \cup (11)^*)^*00$

La concatenazione avviene collegando gli stati finali del precedente automa con lo stato iniziale dell'automata da concatenare tramite ϵ transition. Ricorda che si conserva come stato finale solo quello dell'automata a cui concateniamo.



X1, X2, X3 e start erano i precedenti stati finali collegati con ϵ transition allo stato iniziale dell'automa 00 a cui andava concatenato.

Il procedimento è ridondante sì, ma semplice da eseguire.

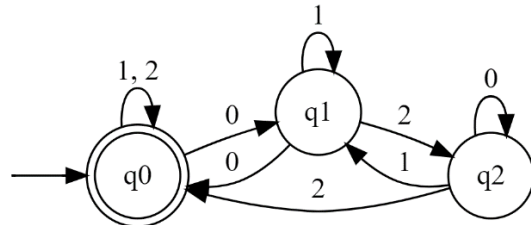


CoScienze
Associazione

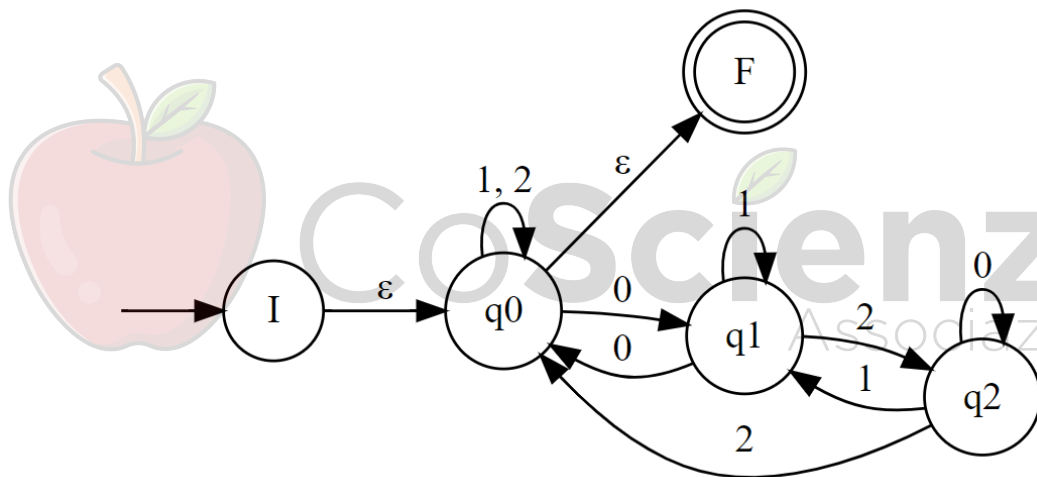
Determinare Espressione Regolare da Automa

	0	1	2
q_0	q_1	q_0	q_0
q_1	q_0	q_1	q_2
q_2	q_2	q_1	q_0

Disegnare l'automa

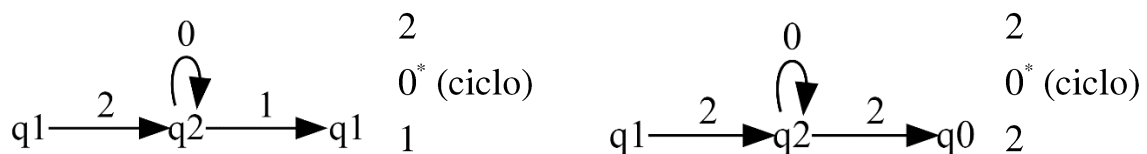


Aggiungere stato iniziale (I) e finale (F) fittizi

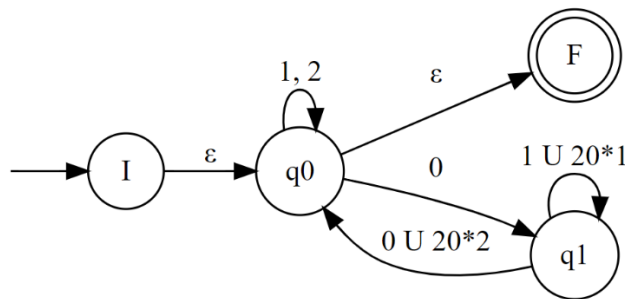


Scegliere uno stato da eliminare, ad esempio q_2 e calcolarne i percorsi.

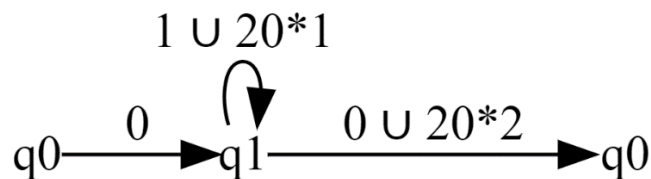
In q_2 si arriva solo da q_1 ed ha due uscite q_1 e q_0 ed una ridondanza; quindi, i percorsi saranno i seguenti



Ed ottengo di conseguenza dal primo percorso un'aggiunta al ciclo su q_1 e dal secondo un'aggiunta al collegamento $q_1 \rightarrow q_0$



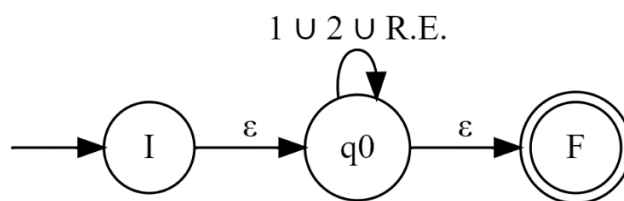
Si reitera il processo scegliendo un nuovo nodo da eliminare, banalmente q_1 , e ne calcolo i percorsi. Questa volta il percorso è solo uno.



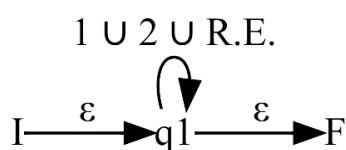
E quindi ottengo un'aggiunta al ciclo su q_0 , dato che l'espressione che ne esce è lunga la riassumo e da qui in avanti sarà chiamata R.E.

$$R.E. = (0(1 \cup 20^*1)^*(0 \cup 20^*2))$$

Di conseguenza ottengo il seguente automa



Non rimane che eliminare l'ultimo stato q_0 , ma dato che qui il passo è banale avendo ϵ transition in entrata e uscita dal nodo e quindi è solo un ciclo, l'unico percorso è



$$(1 \cup 2 \cup R.E.)^*$$

ovvero

$$(1 \cup 2 \cup (0(1 \cup 20^*1)^*(0 \cup 20^*2)))^*$$

Funzione di transizione estesa F^*

La funzione di transizione F può essere estesa a F^* che opera su stati e stringhe.

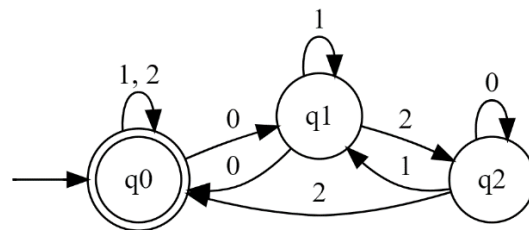
Esercizio del tipo dato l'automa (con stato iniziale q_0 e $F = \{q_0\}$)

- determinare $F^*(q_0, 0122)$

Prendiamo ad esempio il seguente automa.

	0	1	2
q_0	q_1	q_0	q_0
q_1	q_0	q_1	q_2
q_2	q_2	q_1	q_0

Disegnare l'automa



Passo 1

$$F^*(q_0, 0122) = F(F^*(q_0, 012), 2)$$

Passo 2

$$F^*(q_0, 012) = F(F^*(q_0, 01), 2)$$

Passo 3

$$F^*(q_0, 01) = F(F^*(q_0, 0), 1)$$

Passo 4

$$F^*(q_0, 0) = F(F^*(q_0, \varepsilon), 0)$$

Risultato

$$= F(q_2, 2) = q_0$$

$$= F(q_1, 2) = q_2$$

$$= F(q_1, 1) = q_1$$

$$= F(q_0, 0) = q_1$$

Procedimento

In q_0 con input 0 si ottiene lo stato q_1

$$F^*(q_0, 0) = F(F^*(q_0, \varepsilon), 0) = F(q_0, 0) = q_1$$

sostituisco $F^*(q_0, 0)$ con q_1 nel Passo 3

$$F^*(q_0, 01) = F(F^*(q_0, 0), 1) = F(q_1, 1) = q_1$$

sostituisco $F^*(q_0, 01)$ con q_1 nel Passo 2

$$F^*(q_0, 012) = F(F^*(q_0, 01), 2) = F(q_1, 2) = q_2$$

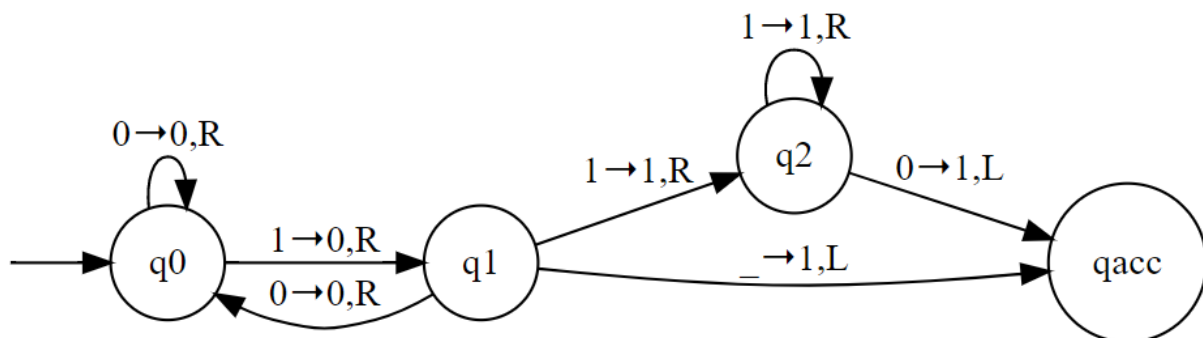
sostituisco $F^*(q_0, 012)$ con q_2 nel Passo 1

$$F^*(q_0, 0122) = F(F^*(q_0, 012), 2) = F(q_2, 2) = q_0$$

Determinare configurazione MdT

In questo tipo di esercizio viene fornita una MdT ed una o più stringhe di cui calcolarne la configurazione.

Prendiamo in esempio il seguente automa e calcoliamo la configurazione per le seguenti stringhe 00110 e 0111



[00110]

Si parte ovviamente da

- $C_1 = q_0 00110 \Rightarrow$ la testina ora punta al “primo” 0

Una volta letto 0 la testina si sposta a destra, lo 0 rimane e lo stato resta q_0

- $C_2 = 0q_0 0110 \Rightarrow$ la testina ora punta al “secondo” 0

Una volta letto 0 la testina si sposta a destra, lo 0 rimane e lo stato resta q_0

- $C_3 = 00q_0 110 \Rightarrow$ la testina ora punta al “primo” 1

Una volta letto 1 la testina si sposta a destra, sostituisce 1 con 0 e lo stato ora è q_1

- $C_4 = 000q_1 10 \Rightarrow$ la testina ora punta al “secondo” 1

Una volta letto 1 la testina si sposta a destra, l'1 rimane e lo stato ora è q_2

- $C_5 = 0001q_2 0 \Rightarrow$ la testina ora punta all'ultimo 0

Una volta letto la testina si sposta a sinistra, sostituisce 0 con 1 e lo stato ora è q_{acc}

- $C_6 = 000q_{acc} 11 \Rightarrow CONF. ACCEPT$

[0111]

Si parte ovviamente da

- $C_1 = q_0 0111 \Rightarrow$ la testina punta al primo 0

Una volta letto 0 la testina si sposta a destra, lo 0 rimane e lo stato resta q_0

- $C_2 = 0q_0 111 \Rightarrow$ la testina ora punta al primo 1

Una volta letto 1 la testina si sposta a destra, sostituisce 1 con 0 e lo stato ora è q_1

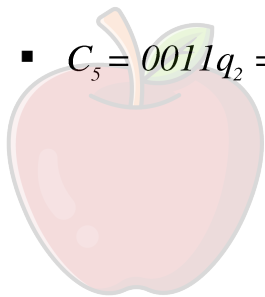
- $C_3 = 00q_1 11 \Rightarrow$ la testina ora punta al “secondo” 1

Una volta letto 1 la testina si sposta a destra, l'1 rimane e lo stato ora è q_2

- $C_4 = 001q_2 1 \Rightarrow$ la testina ora punta al “terzo” 1

Una volta letto 1 la testina si sposta a destra, l'1 rimane e lo stato resta q_2

- $C_5 = 0011q_2 \Rightarrow$ non è stato raggiunto lo stato q_{acc} quindi *CONF. REJECT*



CoScienze
Associazione

Funzioni con MdT

Cominciamo col definire due tipologie di funzioni generiche con MdT.

Le funzioni di conta/calcolo e le funzioni di comparazione.

Definiamo come funzioni di comparazione quelle del tipo

$$f(x) = \begin{cases} 1, & x > y \\ 0, & x \leq y \end{cases}$$

Definiamo come funzioni di conta/calcolo quelle del tipo

$$f(x) = x + 2$$

Funzione di comparazione



$$f(x) = \begin{cases} 1, & x > y \\ 0, & x \leq y \end{cases}$$

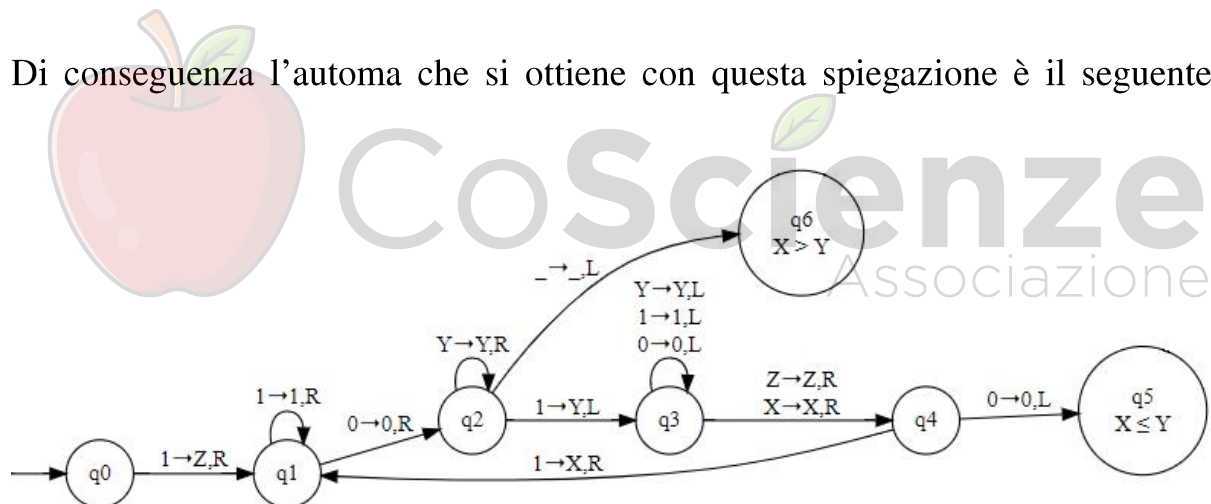
- 1) Marcare con un nuovo carattere (Z) il primo 1 che incontriamo in modo da salvare l'inizio della stringa mi sposto a destra e cambio stato (q_1)
- 2) Ciclo su sé stesso il nuovo stato (q_1) finché incontro 1 spostandomi a destra sulla stringa, quando incontro 0 mi sposto a destra ed in un nuovo stato (q_2) per analizzare il secondo numero
- 3) Quando incontro il primo 1 lo marco con un nuovo carattere (Y) e comincio a scorrere la stringa verso sinistra e mi sposto in un nuovo stato (q_3)
- 4) Ciclo su me stesso (q_3) qualsiasi carattere posso incontrare del tipo 1,0 e Y perché cerco l'inizio della stringa. Quando incontro Z mi sposto in un nuovo stato (q_4) riscrivo Z e mi sposto a destra perché sono all'inizio della stringa
- 5) A questo punto se incontro 0 mi sposto nel nuovo stato q_5 spostando la testina a sinistra e so che la stringa alla destra dello 0 contiene un numero

maggiore o uguale di 1 di quella a sinistra situazione del tipo $ZX0Y1$. Se incontro 1 sovrascrivo con un nuovo carattere (X) mi sposto a destra e vado nello stato (q_1) per scorrere la stringa verso destra come nel punto 2

A questo punto aggiungo le condizioni generate dai nuovi simboli aggiunti:

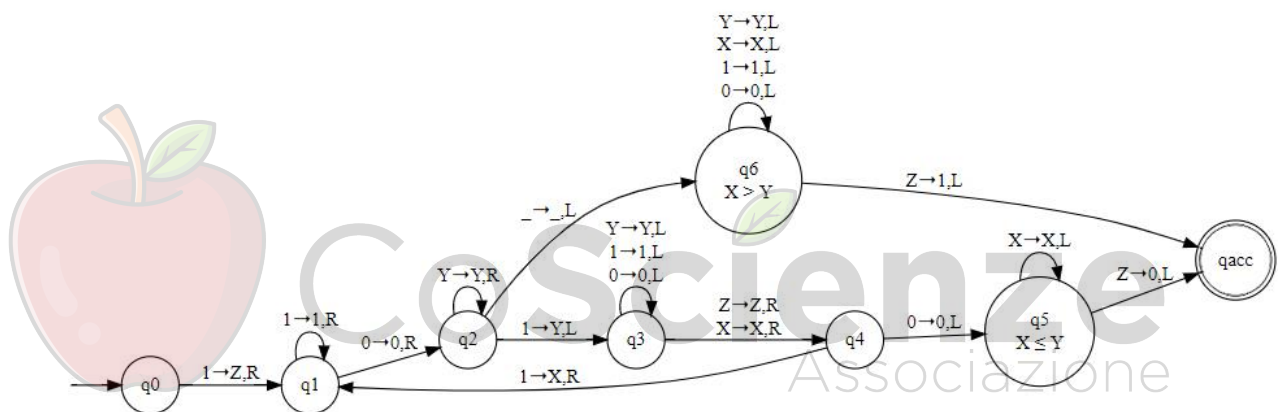
- 6) Ciclo q_2 su sé stesso finché incontro Y riscrivendo Y e spostandomi a destra.
- 7) Se nello stato q_2 leggo un *blank* ($_$) vuol dire che mi trovo in una situazione del genere " $ZXX0YY$ " e quindi la stringa alla sinistra dello 0 contiene un numero maggiore di 1 di quella alla destra, quindi sposto la testina a sinistra e mi sposto in un nuovo stato q_6 .
- 8) Aggiungo lo spostamento a destra leggendo X e scrivendo X nel collegamento $q_3 \rightarrow q_4$ perché potrei trovarmi nella situazione in cui la stringa $ZX10Y11$ e quindi devo ricominciare il ciclo o dedurre una soluzione

Di conseguenza l'automa che si ottiene con questa spiegazione è il seguente:



Calcolata la base del comparatore aggiungiamo la computazione ovvero, se $X > Y$ scrivo 1, se $X \leq Y$ scrivo 0

- 1) Ciclo q_5 su sé stesso spostandomi verso sinistra per spostarmi fino all'inizio della stringa, in questo caso posso solo incontrare delle X. A questo punto quando incontro il carattere Z lo sostituisco con 0 e mi sposto a sinistra essendo arrivati all'inizio la testina punterà sempre a questo carattere e vado nello stato q_{acc}
- 2) Ciclo q_6 su sé stesso spostandomi verso sinistra per spostarmi fino all'inizio della stringa qualsivoglia carattere incontro eccezion fatta per la Z. A questo punto quando incontro il carattere Z lo sostituisco con 1 e mi sposto a sinistra essendo arrivati all'inizio la testina punterà sempre a questo carattere e vado nello stato q_{acc}

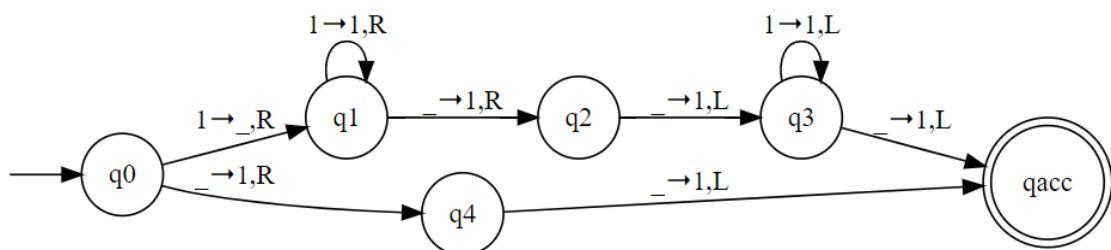


Funzione Somma

$$f(x) = x + 2$$

- 1) Il primo stato q_0 come per il comparatore svolge il ruolo di marker, quindi, appena legge il primo 1 lo sostituisce con il carattere *blank* ($_$) spostando la testina a destra e andando nello stato q_1
- 2) Il secondo stato cicla su sé stesso fino alla fine della stringa quindi se legge 1 scrive 1 e si sposta a destra, se legge *blank* ($_$) scrive 1 allora si sposta a destra andando nello stato q_2 aggiungendo alla stringa il primo 1 richiesto
- 3) Trovandoci nello stato q_2 , l'unico carattere che potremo incontrare sarà il *blank* ($_$). Quindi lo sostituisco con 1 per aggiungere il secondo 1 richiesto, ma mi comincio a spostare verso sinistra con la testina dato che devo ritornare ad inizio stringa e quindi vado nello stato q_3
- 4) In q_3 effettuo un ciclo leggendo 1 scrivendo 1 e spostandomi verso sinistra per tornare ad inizio stringa, quando la macchina legge *blank* ($_$) questo sarà il *blank* scritto al punto 1 dallo stato "marker" quindi lo sostituisco con 1 e mi sposto a sinistra, essendo arrivati ad inizio stringa la testina punterà sempre a questo carattere e vado nello stato q_{acc}
- 5) Per terminare aggiungo il caso stringa vuota ovvero $x + 2$ con $x = 0$. Molto semplicemente, aggiungo un nuovo stato e 2 movimenti speculari a quelli di entrata e uscita di q_2 che terminano direttamente nello stato q_{acc}

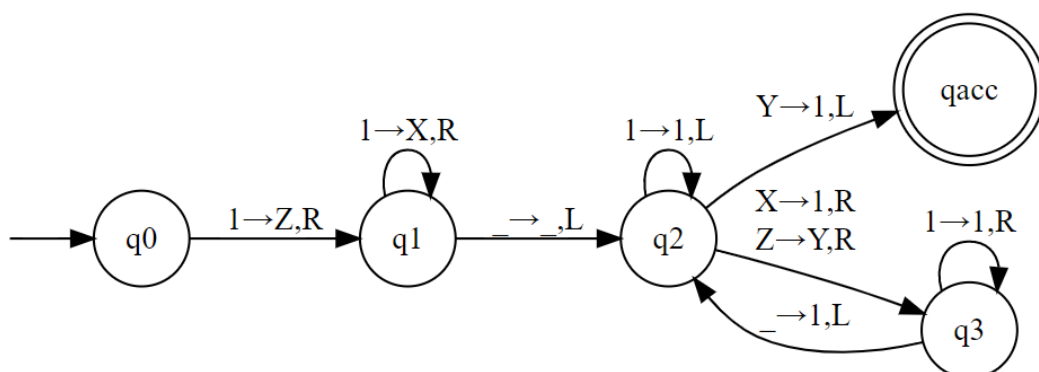
L'automa ottenuto è il seguente



Funzione Multiplo di 2

$$f(x) = 2x$$

- 1) Il primo stato q_0 svolge il ruolo di marker, quindi, appena legge il primo 1 lo sostituisce con il carattere Z spostando la testina a destra e andando nello stato q_1
- 2) Lo stato q_1 converte la restante stringa di 1 in X spostandosi sempre a destra lungo la stringa quando legge il blank () si sposta a sinistra e va nello stato q_2
- 3) Lo stato q_2 è usato per tornare indietro lungo la stringa se incontriamo il carattere 1 perché andiamo alla ricerca dei caratteri Z e X, se incontro X scrivo 1 e mi sposto nello stato q_3 .
- 4) Nello stato q_3 ci spostiamo alla fine della stringa per aggiungere un carattere 1 al posto del primo blank () che incontriamo andando nuovamente in q_2 e spostandoci a sinistra della stringa così il successivo carattere incontrato nello stato q_2 sarà sicuramente un 1. In questo modo nello stato precedente abbiamo inserito un 1 e in questo ne abbiamo inserito un altro alla fine della stringa.
- 5) Il ciclo continua fino alla sostituzione di tutte le X in 1 aggiungendo un 1 al posto del blank () finale, quando incontro Z invece di X scrivo Y che uso come carattere di uscita. Bisogna aggiungere questo ulteriore carattere perché per come è strutturata la macchina una volta incontrato Z devo fare un ultimo ciclo prima di andare in q_{acc} .



Risposte brevi a domande comuni

Dire se le seguenti affermazioni sono vere o false, giustificando le risposte:

Se $L = \emptyset$ allora L è regolare

VERO: è facile esibire un automa che accetti il linguaggio $\emptyset$. L'automa potrà avere l'insieme degli stati finali vuoto o comunque costituito da stati non raggiungibili dallo stato iniziale

Per ogni $a \in \Sigma$, vale la seguente uguaglianza tra espressioni regolari: $a\epsilon = a\emptyset$

FALSO: l'espressione $a\epsilon$ denota il linguaggio $\{a\}$ mentre l'espressione $a\emptyset$ denota il linguaggio $\emptyset$.

Se $A \cap B$ è regolare allora A e B sono regolari

FALSO: è vero il viceversa, cioè che se A e B sono regolari allora $A \cap B$ è regolare. Per provare che l'affermazione è falsa basta un controesempio, p.e. prendendo A e B disgiunti oppure A sottoinsieme regolare di B , o altro...

Se $A \cup B$ non è regolare allora almeno uno tra A e B non è regolare

VERO: è la proprietà di chiusura dei linguaggi regolari rispetto l'unione: se A e B fossero entrambi regolari allora anche $A \cup B$ sarebbe regolare.

Se il complemento di A è regolare allora A è regolare

VERO: è la proprietà di chiusura dei linguaggi regolari rispetto al complemento, infatti A è il complemento del complemento di A .

Ogni linguaggio regolare è decidibile.

VERO: se un linguaggio L è regolare allora esiste un dfa che lo accetta. Poiché il dfa termina per ogni input, una macchina di Turing che si comporta come il dfa decide L .

Ogni linguaggio decidibile è Turing riconoscibile

VERO: se un linguaggio è decidibile allora esiste una macchina di Turing che lo riconosce (e termina per ogni input). Quindi il linguaggio è certamente anche Turing riconoscibile.

Se un linguaggio è Turing riconoscibile allora il suo complemento non è Turing riconoscibile.

FALSO: infatti anche i linguaggi decidibili sono Turing riconoscibili, e il complemento di un linguaggio decidibile è ancora decidibile (basta invertire le risposte della macchina di Turing che decide il linguaggio dato: Accept diventa Reject e viceversa). L'affermazione è vera solo per linguaggi non decidibili.

Se L_1 è Turing riconoscibile e non decidibile e L_1 si riduce a L_2 allora anche L_2 è Turing riconoscibile e non decidibile.

FALSO: possiamo concludere che L_2 non è decidibile ma potrebbe anche essere non Turing riconoscibile (bisognerebbe esibire un controesempio, cioè di un linguaggio Turing riconoscibile che si riduce ad un linguaggio che non è Turing riconoscibile)

Data una macchina di Turing M , la proprietà “ M termina in almeno 10 mosse” è decidibile;

VERO: basta costruire una macchina che prende in input M e la simula (in maniera analoga alla macchina universale che riconosce A_{TM}) ma solo per 10 passi. Quindi questa macchina termina sempre (cosa che invece la macchina universale potrebbe non fare). Osservazione: non centra niente il teorema di Rice perché la proprietà “ M termina in almeno 10 mosse” **non** è una proprietà del linguaggio accettato da M .

Se $A \notin P$ allora neanche il complemento di A è in P

VERO: se A fosse in P lo sarebbe anche il suo complemento (vedi anche risposta al punto h)

Se $A \in NP$ allora anche il complemento di A appartiene a NP

NON NOTO. (se $P=NP$ sarebbe vero)

Se $P \neq NP$ allora $P \cap NP = \emptyset$

FALSO: $P \cap NP = P$ poiché $P \subseteq NP$. Se $P \neq NP$ sappiamo solo che l'inclusione è stretta ma resta $P \cap NP = P$.

Se A è NP completo e $A \leq_p B$ allora anche B è NP completo

FALSO: sarebbe vero se sapessimo che B è in NP

Sia A è NP completo. A è in P se e solo se $NP=P$.

VERO: Supponiamo che $A \in P$. Poiché A è NP -completo, sappiamo che ogni problema in NP si riduce ad A e quindi ogni problema in NP sarebbe anche in P e quindi $P=NP$ (perché sappiamo già che $P \subseteq NP$ e quindi basta provare che $NP \subseteq P$). Viceversa, se $P=NP$ certamente $A \in P$, poiché $A \in NP$ per definizione di NP -completezza